Lab 09 — Compose: de stack orkestreren

Theorie — hoe je drie `podman run`-commando's van tien regels vervangt door één declaratief bestand, de diensten in de juiste volgorde laat starten, en hetzelfde bestand gebruikt van ontwikkelmachine tot acceptatie.

Doelstellingen

- Begrijpen wat Compose toevoegt ten opzichte van scripts met `podman run`, en wat het niet doet.
- Een `compose.yaml` lezen en schrijven: `services`, `volumes`, `networks`, `secrets`, `depends_on`, `healthcheck`.
- De cyclus `up` / `ps` / `logs` / `exec` / `down` beheersen, en weten wat `down` vernietigt of bewaart.
- Meerdere omgevingen beheren: `compose.override.yaml`, `-f`, `.env`, `podman compose config`.
- Weten wat `podman compose` werkelijk uitvoert, en waarin het verschilt van Docker Compose.

---

1. Het probleem: de stack past in drie commando's… van tien regels

In lab 07 vroeg de keten `web → api → db` een `podman network create`, een volume, drie `podman run`-commando's met hun `-e`, `-v`, `-p`, `--secret`, in de juiste volgorde, en daarna het omgekeerde om op te ruimen. Elke collega typt die commando's net iets anders; niemand weet welke versie draait; en de dag dat er een dienst bijkomt, zit iedereen in zijn shellgeschiedenis te zoeken.

Compose vervangt dat alles door één bestand, `compose.yaml`, dat de gewenste toestand **beschrijft**: de diensten, hun images, hun variabelen, hun volumes, hun afhankelijkheden. Eén commando, `podman compose up -d`, maakt aan wat ontbreekt; een ander, `podman compose down`, ruimt alles netjes op. Het bestand wordt samen met de code geversioneerd: de stack wordt reproduceerbaar.

> **→ GESCHIEDENIS**
> In 2014 publiceert een klein bedrijf, Orchard, **Fig**, een Python-tool om meerdere Docker-containers in een YAML-bestand te beschrijven. Docker neemt Orchard datzelfde jaar over; Fig wordt `docker-compose` (2015). In 2020 publiceert Docker de **Compose Specification**, een open standaard, en herschrijft de tool in Go: `docker compose` (v2, zonder koppelteken). `podman-compose`, sinds 2019 in Python geschreven, implementeert diezelfde specificatie bovenop Podman.

2. Anatomie van een `compose.yaml`

```yaml
name: stack                                  # projectnaam (anders: de map)
services:
  db:
    image: docker.io/library/postgres:16-alpine
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
    volumes:
      - pgdata:/var/lib/postgresql/data      # named volume (verderop gedeclareerd)
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5
  api:
    image: localhost/api:${API_TAG:-1.0}     # gesubstitueerd vanuit .env
    build:
      context: ../05                         # of gebouwd vanuit deze Dockerfile
    env_file: [api.env]
    depends_on:
      db:
        condition: service_healthy           # wacht tot db "healthy" is
  web:
    image: docker.io/library/nginx:alpine
    ports: ["18080:80"]
    volumes:
      - ./nginx-proxy.conf:/etc/nginx/conf.d/default.conf:ro   # bind mount
    depends_on:
      api:
        condition: service_healthy
volumes:
  pgdata:
secrets:
  db_password:
    file: ./db_password.txt
```

Elke sleutel onder `services` komt overeen met een optie van `podman run` die je al kent: `image` / `build`, `ports` (`-p`), `environment` (`-e`), `env_file` (`--env-file`), `volumes` (`-v`), `secrets` (`--secret`), `restart` (`--restart`), `healthcheck` (`HEALTHCHECK`), `command` / `entrypoint`. Drie dingen zijn **impliciet**: er wordt een **netwerk** `<project>_default` aangemaakt waarop alle diensten aangesloten zijn; elke dienst is er bereikbaar op **zijn naam** (de DNS van lab 07); en de containers heten `<project>_<dienst>_1`.

→ YAML
YAML steunt op **inspringing** (twee spaties, nooit een tab): `db:` onder `services:` is een dienst, `pgdata:` onder `volumes:` is een volume. Een waarde zonder iets na de dubbele punt (`pgdata:`) is leeg: "maak het aan met de standaardopties". Lijsten schrijf je met `-` of tussen vierkante haken; strings die een `:` bevatten (zoals `"18080:80"`) zet je tussen aanhalingstekens, anders ziet YAML er een sleutel in. `podman compose config` toont het bestand zoals de tool het begrepen heeft: de eerste reflex wanneer "het niet doet wat ik geschreven heb".

3. De levenscyclus

Commando	Effect	Opmerking
`podman compose up -d`	maakt netwerk, volumes, secrets, containers aan; start in de volgorde van `depends_on`	opnieuw gestart, maakt alleen opnieuw aan wat veranderd is
`podman compose ps`	toestand van de diensten van het project	`podman ps` toont dezelfde containers
`podman compose logs -f api`	logs van één of alle diensten	voorafgegaan door `[api] |`
`podman compose exec db psql …`	commando in een dienst	`podman exec stack_db_1 …` doet hetzelfde
`podman compose stop` / `start` / `restart`	zonder te vernietigen	zoals `podman stop/start`
`podman compose down`	stopt en verwijdert containers, netwerk, pod	**bewaart de volumes**
`podman compose down -v`	idem, **plus de volumes**	de database verdwijnt
`podman compose build`	bouwt de diensten die `build:` hebben	`up --build` doet beide
`podman compose config`	toont de samengevoegde en gesubstitueerde configuratie	lezen vóór `up`

De **projectnaam** is de sleutel tot alles: ze is het voorvoegsel van containers, netwerk en volumes. Standaard is het de naam van de map; `name:` in het bestand (of `-p`) legt ze vast. Hetzelfde bestand starten vanuit een hernoemde map maakt **een tweede project** aan, met een leeg volume `andere_pgdata`: "de database is verdwenen" is het klassieke symptoom.

4. In de juiste volgorde starten

`depends_on: [db]` alleen garandeert dat `db` **gestart** is vóór `api` — niet dat het klaar is. PostgreSQL heeft twee seconden nodig om verbindingen te aanvaarden; Spring Boot probeert meteen te verbinden en stopt met `Connection refused`. Vandaar de lange vorm: `condition: service_healthy`, die wacht tot de **healthcheck** van `db` slaagt.

Een healthcheck is een commando dat op regelmatige tijdstippen **in** de container uitgevoerd wordt; zijn exitcode beslist: `0` gezond, iets anders ziek. Na `retries` opeenvolgende mislukkingen wordt de toestand `unhealthy`. Hij kan uit de Dockerfile komen (`HEALTHCHECK`, lab 10) of uit de `compose.yaml`, die voorrang heeft. Het commando moet **in de image bestaan**: `curl` zit niet in `eclipse-temurin:21-jre-alpine`, `wget` (BusyBox) wel.

→ SPRING BOOT
Een geordende start ontslaat de applicatie van niets: de database kan **achteraf** uitvallen. Een robuuste dienst probeert opnieuw (`spring.datasource.hikari.initializationFailTimeout=-1` start zonder database en wacht), en `/actuator/health` wordt de natuurlijke healthcheck van de API — daarop wacht `web` in onze stack. In Kubernetes (lab 12) heten diezelfde controles *readiness* en *liveness probes*.

PODMAN
In rootless voert Podman de healthchecks uit met **systemd-timers**: zonder systemd-sessie (een WSL-distributie zonder `systemd=true`) blijft de toestand eeuwig `starting` , en blokkeert `condition: service_healthy` de `up` . Controleer `systemctl --user status` voor je Compose de schuld geeft. Docker plant de healthchecks in zijn daemon.

5. Eén bestand, meerdere omgevingen

Drie mechanismen werken samen:

- `compose.override.yaml` wordt **automatisch** samengevoegd met `compose.yaml` : daar zet je wat alleen in ontwikkeling geldt (de database publiceren op `127.0.0.1:5432` , de broncode mounten, een profiel `dev`).
- `-f compose.yaml -f compose.prod.yaml` voegt de opgegeven bestanden expliciet samen, in die volgorde, en **negeert dan de override**: `restart: unless-stopped` , geen poort voor de database, profiel `prod` .
- `.env` naast het bestand levert de variabelen die **in de YAML gesubstitueerd** worden: `${API_TAG:-1.0}` is `API_TAG` als die bestaat, anders `1.0` . `API_TAG=2.0 podman compose config` toont het.

VALKUIL
`.env` en `env_file` hebben **niets** met elkaar te maken. `.env` wordt door Compose gelezen om de `${…}` in het YAML-bestand te vervangen; het gaat de containers niet in. `env_file: [api.env]` wordt aan de container doorgegeven als `--env-file` . Een `APP_MESSAGE=…` in `.env` bereikt de API nooit. Tweede valkuil: `podman compose exec` leest de standaardbestanden opnieuw om de omgeving van de dienst in te spuiten — is het project met `-f` gestart, geef dan **dezelfde** `-f` aan `exec` (of gebruik `podman exec`), anders zie je de variabelen van de override.

6. Wat `podman compose` uitvoert

PODMAN
`podman compose` is maar een **doorgeefluik**: het zoekt `podman-compose` (Python, `apt install podman-compose` ; neem een versie ≥ 1.1, via `pipx` als die van `apt` oud is) of `docker-compose` , en geeft het commando door — vandaar de banner `>>>> Executing external compose provider` . Zichtbare verschillen met Docker Compose: de containers heten `stack_api_1` (Docker: `stack-api-1`); podman-compose groepeert de containers in een **pod** `pod_stack` zonder infra-container (een loutere groepering, elke dienst houdt zijn IP); `down` kan de instanties van `--scale` niet verwijderen; en `restart:` wordt door de container toegepast, zonder daemon om hem na `wsl --shutdown` opnieuw te starten (lab 03). Voor een echte automatische start: Quadlet (lab 10). De andere weg, dichter bij Docker: `systemctl --user enable --now podman.socket` en de echte `docker compose` v2 gericht op `DOCKER_HOST=unix://$XDG_RUNTIME_DIR/podman/podman.sock` .

7. In het bedrijf

- **Compose is de tool van het werkstation en de CI**: één `podman compose up -d` geeft elke ontwikkelaar de volledige stack in één minuut, en de CI gebruikt het voor integratietests (`up -d`, tests, `down -v`).
- **Het past bij kleine uitrollen**: één VM, één applicatie, `compose.prod.yaml` en `restart: unless-stopped`. Het biedt geen hoge beschikbaarheid, geen update zonder onderbreking, geen spreiding over meerdere machines: dat is de grens met Kubernetes (lab 11).
- **Alles wat je hier geschreven hebt, is overzetbaar**: een dienst wordt een Deployment, een named volume een PersistentVolumeClaim, een healthcheck een *probe*, `depends_on` een *readiness*, `ports` een Service. Compose is de kladversie van je manifesten.
- **De secrets** blijven buiten het bestand: `secrets:` wijst naar een bestand dat Git negeert, of naar een secret van de engine (`external: true`).

Onthouden

- Compose beschrijft de gewenste toestand in een geversioneerd bestand; `up` maakt aan wat ontbreekt, `down` verwijdert alles behalve de volumes.
- Een dienst = een `podman run` vertaald naar YAML; netwerk en DNS zijn impliciet, op dienstnaam.
- De projectnaam is het voorvoegsel van alles; `name:` vermijdt spookprojecten en "verdwenen" volumes.
- `depends_on` alleen wacht op het starten; `condition: service_healthy` wacht op een geslaagde healthcheck — met een commando dat in de image zit.
- `compose.override.yaml` is automatisch, `-f` is expliciet en negeert ze; `podman compose config` toont het resultaat.
- `.env` substitueert in de YAML, `env_file` gaat de container in: twee verschillende dingen.
- `podman compose` geeft door aan podman-compose: namen met `_`, een groeperingspod, geen `scale` naar beneden, `restart` zonder daemon.

Woordenschat

Compose: tool en formaat om een applicatie met meerdere containers te beschrijven. — **project**: benoemde verzameling resources aangemaakt vanuit een Compose-bestand. — **dienst**: een container (of meerdere instanties) beschreven onder `services:`. — **`depends_on`**: startvolgorde, met of zonder voorwaarde. — **healthcheck**: periodiek commando dat de toestand `healthy`/`unhealthy` bepaalt. — **override**: bestand dat bovenop `compose.yaml` samengevoegd wordt. — **substitutie**: vervanging van `${VAR}` vanuit `.env` of de shell. — **`podman compose config`**: weergave van de samengevoegde configuratie. — **podman-compose**: Python-implementatie van de Compose Specification voor Podman. — **projectpod**: groepering `pod_<project>` aangemaakt door podman-compose.